



ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ

Повторение линейной алгебры и сходимость последовательностей

МЕТОДЫ ОПТИМИЗАЦИИ

НЕДЕЛЯ 1

План лекции



1. Числа: $\mathbb{R}$ vs floating point; GPU-форматы
2. Векторы: операции и нормы
3. Матрицы: операции и нормы
4. Тензоры: что это и зачем
5. Разложения матриц: Холецкий, QR, спектральное, SVD
6. Разложения тензоров: CP, Tucker, TT

Вещественные числа и компьютерная арифметика

- В теории: $x \in \mathbb{R}$, точные операции $+$, $\cdot$, $\dots$
- На компьютере: конечное множество чисел с плавающей точкой
- Модель IEEE 754 (типично `float64/float32`)
- Следствие: округления, переполнения, потеря значимости

Представление floating point

Число записывается как

$$x = (-1)^s \cdot m \cdot 2^e,$$

где

- s — знак
- m — мантисса (значимая часть)
- e — порядок (экспонента)

Ключевые параметры формата: точность мантиссы и диапазон e .

Машинная точность $\varepsilon_{\text{mach}}$

- $\varepsilon_{\text{mach}}$ — наименьшее $\varepsilon > 0$, такое что $\text{fl}(1 + \varepsilon) \neq 1$.
- Относительная ошибка представления: порядка $\varepsilon_{\text{mach}}$.
- Зависит от числа бит мантиссы, не от экспоненты.

Классические форматы: fp64 и fp32



Формат	Биты	Exp	Mantissa	$\varepsilon_{\text{mach}}$
fp64 (double)	64	11	52	$\approx 2 \cdot 10^{-16}$
fp32 (float)	32	8	23	$\approx 1 \cdot 10^{-7}$

- На CPU научный код часто в fp64
- На GPU обучение нейросетей — в основном fp32 и ниже

GPU-форматы: зачем снижать точность



- Больше throughput (Tensor Cores): быстрее matmul
- Меньше память и ширина шины: больше batch/модель
- Trade-off: скорость vs численная устойчивость
- Практика: mixed precision — низкая точность в matmul, накопление в более высокой

fp16 и bf16



Формат	Биты	Exp	Mantissa	Комментарий
fp16 (half)	16	5	10	IEEE 754 binary16
bf16 (bfloat16)	16	8	7	«урезанный» fp32

- fp16: лучше точность, но узкий диапазон $\Rightarrow$ риск overflow/underflow.
- bf16: тот же диапазон, что у fp32, хуже точность.
- В deep learning чаще предпочитают bf16.

Другие популярные форматы на GPU



- tf32 (TensorFloat-32, NVIDIA): 8 бит exp + 10 бит mantissa; диапазон как у fp32, точность ближе к fp16
- fp8: 8 бит; обучение и inference LLM
- fp4/микроскейлинг: ещё агрессивнее сжатие (новые GPU)
- Целочисленные: int8, int4 — в основном quantize/inference

Сравнение форматов (схема)



Формат	Диапазон	Точность	Типичное применение
fp64	огромный	очень высокая	научный HPC
fp32	большой	высокая	baseline на GPU
tf32	как fp32	средняя	ускоренный GEMM
bf16	как fp32	ниже	обучение DL
fp16	узкий	средняя	mixed precision
fp8	очень узкий	низкая	LLM train/infer
int8/4	—	—	квантизация

Mixed precision на практике

- Веса/активации/градиенты — в fp16/bf16.
- Master-копия весов и накопители — в fp32.
- Loss scaling (для fp16): защищает мелкие градиенты от underflow.
- Для bf16 loss scaling часто не нужен.

$\mathbb{R}$ vs floating point: в чём разница



- Ассоциативность не обязана выполняться: $(a + b) + c \neq a + (b + c)$ в FP.
- Вычитание близких чисел $\Rightarrow$ потеря значимости.
- a/b и $\sqrt{a}$ могут быть «дорогими» и неточными.
- Накопление ошибок в длинных итерациях оптимизации.
- На GPU ошибки сильнее из-за низкоточных форматов.

Почему это важно в оптимизации

- Критерии остановки: $\|x_{k+1} - x_k\| < \tau$.
- Слишком маленький τ — бессмысленен из-за FP.
- Плохо обусловленные задачи усиливают ошибки округления.
- Низкая точность (fp16/fp8) может ломать сходимость или требовать других гиперпараметров.
- Разложения (Холецкий, QR, SVD) — считать устойчивее (обычно в fp32/fp64).

Векторы



По умолчанию вектор — столбец:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n, \quad \mathbf{x}^\top = [x_1 \quad \cdots \quad x_n].$$

Базовые операции: $\mathbf{x} + \mathbf{y}$, $\alpha \mathbf{x}$, скалярное произведение $\mathbf{x}^\top \mathbf{y}$.

Скалярное произведение

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i.$$

- Линейность по каждому аргументу
- $\langle \mathbf{x}, \mathbf{x} \rangle = \|\mathbf{x}\|_2^2$
- Связь с углом: $\langle \mathbf{x}, \mathbf{y} \rangle = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos \theta$

Норма вектора

Норма $\| \cdot \|$ удовлетворяет:

1. $\|\alpha \mathbf{x}\| = |\alpha| \|\mathbf{x}\|$
2. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$
3. $\|\mathbf{x}\| = 0 \Rightarrow \mathbf{x} = \mathbf{0}$

Расстояние: $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$.

p -нормы

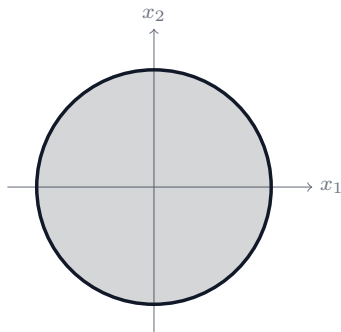


$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \quad p \geq 1.$$

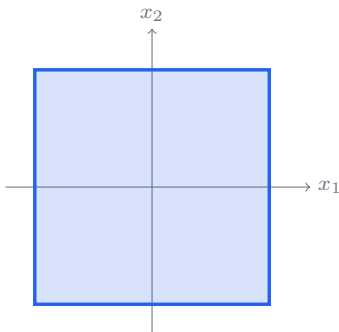
Важные частные случаи:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|, \quad \|\mathbf{x}\|_2 = \sqrt{\sum_i |x_i|^2}, \quad \|\mathbf{x}\|_\infty = \max_i |x_i|.$$

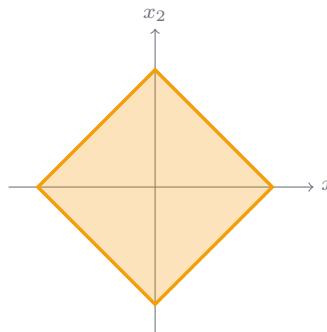
Единичные шары $\{\mathbf{x} : \|\mathbf{x}\|_p \leq 1\}$



$$\|\mathbf{x}\|_2 \leq 1$$



$$\|\mathbf{x}\|_\infty \leq 1$$



$$\|\mathbf{x}\|_1 \leq 1$$

Эквивалентность норм на $\mathbb{R}^n$



- Все нормы на $\mathbb{R}^n$ эквивалентны:

$$c_1 \|\mathbf{x}\|_a \leq \|\mathbf{x}\|_b \leq c_2 \|\mathbf{x}\|_a.$$

- Константы зависят от n и выбора норм.
- В алгоритмах выбор нормы влияет на геометрию шаров и на вид регуляризации ($\ell_1, \ell_2, \dots$).

Матрицы



$$\mathbf{A} \in \mathbb{R}^{m \times n}, \quad \mathbf{A}^{\top} \in \mathbb{R}^{n \times m}.$$

- Нулевая матрица $\mathbf{O}$: все элементы равны 0
- Единичная матрица $\mathbf{I}$: $I_{ij} = \delta_{ij}$ (квадратная; $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$)
- Симметричная: $\mathbf{A} = \mathbf{A}^{\top}$, множество $\mathbb{S}^n$
- Ортогональная: $\mathbf{Q}^{\top} \mathbf{Q} = \mathbf{I}$ (столбцы — ортонормированный базис)

Симметричная положительно определённая матрица

Определение

Матрица $A \in \mathbb{S}^n$ называется *симметричной положительно определённой* (SPD), если

$$x^T A x > 0 \quad \text{для всех } x \neq 0.$$

Обозначение: $A \succ 0$, множество $\mathbb{S}_{++}^n$.

Аналогично $A \succeq 0$ (SPSD), если $x^T A x \geq 0$ для всех x (множество $\mathbb{S}_+^n$).

Базовые операции

- Matvec: $\mathbf{z} = \mathbf{A}\mathbf{x}$, $z_i = \sum_k a_{ik}x_k — \mathcal{O}(mn)$
- Matmul: $\mathbf{C} = \mathbf{AB}$, $c_{ij} = \sum_k a_{ik}b_{kj} — \mathcal{O}(mnp)$
- $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$, но $\mathbf{AB} \neq \mathbf{BA}$ в общем случае
- $\langle \mathbf{x}, \mathbf{Ay} \rangle = \langle \mathbf{A}^\top \mathbf{x}, \mathbf{y} \rangle$
- След: $\text{tr}(\mathbf{A}) = \sum_{i=1}^n a_{ii}$ (для $\mathbf{A} \in \mathbb{R}^{n \times n}$)

Ранг матрицы

Определение

Ранг матрицы $\mathbf{A} \in \mathbb{R}^{m \times n}$ — размерность её пространства столбцов:

$$\text{rank}(\mathbf{A}) = \dim\{\mathbf{A}\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\}.$$

Эквивалентно: максимальное число линейно независимых столбцов (или строк) $\mathbf{A}$.

Теорема о ранге



Теорема

Для любой $\mathbf{A} \in \mathbb{R}^{m \times n}$

$$\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A}^\top) = \text{rank}(\mathbf{A}^\top \mathbf{A}) = \text{rank}(\mathbf{A} \mathbf{A}^\top).$$

Кроме того,

$$\text{rank}(\mathbf{A} \mathbf{B}) \leq \min\{\text{rank}(\mathbf{A}), \text{rank}(\mathbf{B})\}.$$

Матричные нормы

Фробениусова норма:

$$\|\mathbf{A}\|_F = \left(\sum_{i,j} |a_{ij}|^2 \right)^{1/2} = \sqrt{\text{tr}(\mathbf{A}^\top \mathbf{A})}.$$

Спектральная (2-норма, операторная):

$$\|\mathbf{A}\|_2 = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{Ax}\|_2}{\|\mathbf{x}\|_2} = \sigma_{\max}(\mathbf{A}).$$

Скалярное произведение матриц



$$\langle \mathbf{X}, \mathbf{Y} \rangle = \text{tr}(\mathbf{X}^\top \mathbf{Y}) = \sum_{i, j} X_{ij} Y_{ij}.$$

Связь с нормой Фробениуса:

$$\|\mathbf{A}\|_F = \sqrt{\langle \mathbf{A}, \mathbf{A} \rangle}.$$

Число обусловленности

Для невырожденной $\mathbf{A}$:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\|.$$

В спектральной норме:

$$\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}.$$

Для $\mathbf{A} \succ 0$: $\kappa_2(\mathbf{A}) = \frac{\lambda_{\max}(\mathbf{A})}{\lambda_{\min}(\mathbf{A})}.$

Что такое тензор

- Вектор — тензор порядка 1: $\mathbf{x} \in \mathbb{R}^n$
- Матрица — тензор порядка 2: $\mathbf{A} \in \mathbb{R}^{m \times n}$
- Тензор порядка d : $\mathcal{X} \in \mathbb{R}^{n_1 \times \dots \times n_d}$
- Элемент: $\mathcal{X}_{i_1, \dots, i_d}$

Зачем тензоры в оптимизации и ML



- Многомерные данные: изображения, видео, эмбединги
- Параметры моделей как многомерные массивы
- Сжатие и ускорение через низкоранговые структуры
- Дальше: разложения CP/Tucker/TT

Базовые операции с тензорами

- Сложение и умножение на скаляр — поэлементно
- Свёртка / mode- k произведение на матрицу
- Unfolding (матрицизация) по моде k : тензор $\rightarrow$ матрица
- Норма Фробениуса: $\|\mathcal{X}\|_F = \sqrt{\sum |\mathcal{X}_{i_1 \dots i_d}|^2}$

Зачем нужны разложения

- Решать $Ax = b$ устойчивее и быстрее
- Анализировать обусловленность и спектр
- Строить проекции, предобуславливатели, малоранговое приближение
- Стандартные разложения: Холецкий, QR, спектральное, SVD

Разложение Холецкого

Если $\mathbf{A} \succ 0$, то

$$\mathbf{A} = \mathbf{L}\mathbf{L}^\top,$$

где $\mathbf{L}$ — нижнетреугольная с положительной диагональю.

- Сложность: $\mathcal{O}(n^3)$, константа меньше, чем у LU
- Применение: решение $\mathbf{A}\mathbf{x} = \mathbf{b}$, проверка $\mathbf{A} \succ 0$
- В оптимизации: ньютоновские системы, ковариации, ядра

QR-разложение

Для $\mathbf{A} \in \mathbb{R}^{m \times n}$ ($m \geq n$, полный столбцовый ранг):

$$\mathbf{A} = \mathbf{Q}\mathbf{R},$$

где $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, $\mathbf{R}$ — верхнетреугольная.

- Метод наименьших квадратов: $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2$
- Ортогонализация базиса
- Устойчивее нормальных уравнений $\mathbf{A}^\top \mathbf{Ax} = \mathbf{A}^\top \mathbf{b}$

Собственные значения и векторы



Пусть $\lambda \in \mathbb{C}$, $\mathbf{q} \neq \mathbf{0}$:

$$\mathbf{A}\mathbf{q} = \lambda\mathbf{q}.$$

- Для $\mathbf{A} = \mathbf{A}^\top$: все λ вещественные, $\mathbf{q}$ можно выбрать ортонормированными
- $\mathbf{A} \succ 0 \Leftrightarrow$ все $\lambda_i > 0$ (для $\mathbf{A} = \mathbf{A}^\top$)
- $\text{tr } \mathbf{A} = \sum_i \lambda_i$, $\det \mathbf{A} = \prod_i \lambda_i$

Спектральное разложение

Для $\mathbf{A} \in \mathbb{S}^n$:

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top, \quad \mathbf{Q}^\top\mathbf{Q} = \mathbf{I}, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n).$$

Соотношение Рэлея:

$$\lambda_{\min}(\mathbf{A}) = \inf_{\mathbf{x} \neq \mathbf{0}} \frac{\mathbf{x}^\top \mathbf{A} \mathbf{x}}{\mathbf{x}^\top \mathbf{x}}, \quad \lambda_{\max}(\mathbf{A}) = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\mathbf{x}^\top \mathbf{A} \mathbf{x}}{\mathbf{x}^\top \mathbf{x}}.$$

SVD



Для $\mathbf{A} \in \mathbb{R}^{m \times n}$ ранга r :

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top,$$

$$\sigma_1 \geq \dots \geq \sigma_r > 0.$$

- $\|\mathbf{A}\|_2 = \sigma_1, \|\mathbf{A}\|_F^2 = \sum_i \sigma_i^2$
- Связь: $\sigma_i(\mathbf{A}) = \sqrt{\lambda_i(\mathbf{A}^\top \mathbf{A})}$

Теорема Экарта — Янга

Теорема (Eckart — Young — Mirsky)

Пусть $\mathbf{A} \in \mathbb{R}^{m \times n}$ имеет SVD $\mathbf{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, $\sigma_1 \geq \dots \geq \sigma_r > 0$, и $k < r$. Обозначим усечённое SVD

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top.$$

Тогда для нормы $\|\cdot\| \in \{\|\cdot\|_2, \|\cdot\|_F\}$

$$\|\mathbf{A} - \mathbf{A}_k\| = \min_{\substack{\mathbf{B} \in \mathbb{R}^{m \times n} \\ \text{rank}(\mathbf{B}) \leq k}} \|\mathbf{A} - \mathbf{B}\|.$$

Явно: $\|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}$, $\|\mathbf{A} - \mathbf{A}_k\|_F = \left(\sum_{i=k+1}^r \sigma_i^2\right)^{1/2}$.

Сравнение матричных разложений



Разложение	Когда	Вид	Зачем
Холецкий	$A \succ 0$	LL^T	решить $Ax = b$
QR	полный столбц. ранг	QR	least squares
Спектральное	$A = A^T$	$Q\Lambda Q^T$	спектр, квадр. формы
SVD	любая A	$U\Sigma V^T$	ранг, нормы, сжатие

От матриц к тензорам

- У матриц «ранг» однозначен; у тензоров несколько понятий.
- Цель та же: сжать, ускорить, выделить структуру.
- Три классики: CP, Tucker, Tensor Train (TT).

CP-разложение (CANDECOMP/PARAFAC)

$$\mathcal{X} \approx \sum_{r=1}^R \mathbf{a}_r^{(1)} \circ \dots \circ \mathbf{a}_r^{(d)}, \quad \mathcal{X}_{i_1, \dots, i_d} = \sum_{r=1}^R \prod_{k=1}^d A_{i_k r}^{(k)}.$$

- $\mathbf{A}^{(k)} \in \mathbb{R}^{n_k \times R}$ — факторные матрицы; R — CP-ранг
- Хранение: $\mathcal{O}(R \sum_k n_k)$ вместо $\prod_k n_k$
- Минус: CP-ранг считать сложно; разложение может быть неединственным

Разложение Таккера

$$\mathcal{X} \approx \mathcal{G} \times_1 \mathbf{U}^{(1)} \cdots \times_d \mathbf{U}^{(d)}.$$

Покомпонентно:

$$\mathcal{X}_{i_1, \dots, i_d} = \sum_{r_1, \dots, r_d} \mathcal{G}_{r_1, \dots, r_d} U_{i_1 r_1}^{(1)} \cdots U_{i_d r_d}^{(d)}.$$

- Ранги Таккера $(R_1, \dots, R_d); \mathbf{U}^{(k)} \in \mathbb{R}^{n_k \times R_k}$
- Гибче CP, но ядро порядка d плохо масштабируется

Tensor Train (TT)

Ядра $\mathcal{G}_k \in \mathbb{R}^{r_{k-1} \times n_k \times r_k}$ ($r_0 = r_d = 1$):

$$\mathcal{X}_{i_1, \dots, i_d} = \mathbf{G}_1(i_1) \mathbf{G}_2(i_2) \cdots \mathbf{G}_d(i_d)$$

или

$$\sum_{\alpha_1, \dots, \alpha_{d-1}} \mathcal{G}_1(1, i_1, \alpha_1) \mathcal{G}_2(\alpha_1, i_2, \alpha_2) \cdots \mathcal{G}_d(\alpha_{d-1}, i_d, 1).$$

- TT-ранги $(r_1, \dots, r_{d-1})$; хранение $\sim \mathcal{O}(dnr^2)$
- Хорошо масштабируется с порядком d

CP/Tucker/TT: сравнение



	CP	Tucker	TT
Ранг	скаляр R	$(R_1, \dots, R_d)$	$(r_1, \dots, r_d - 1)$
Хранение	$R \sum n_k$	ядро + $\sum n_k R_k$	$\sim dnr^2$
Большой d	ок	тяжело (ядро)	хорошо
Интерпретация	сумма лучей	ядро + базисы	цепочка ядер

Что запомнить

- Floating point $\neq \mathbb{R}$: следим за обусловленностью и точностью.
- На GPU: fp16/bf16/tf32/fp8 — скорость ценой численной устойчивости.
- Нормы задают геометрию пространства и критерии сходимости.
- Разложения матриц — базовый инструмент устойчивых вычислений.
- Тензорные разложения — сжатие многомерных объектов.

Дальше



- Матричное дифференцирование
- Автоматическое дифференцирование
- Одномерная оптимизация

